

FLEETMIND

AI 船舶效能分析與 節能決策支援系統

數字來自計算 語言來自 AI 決策留給人

陽明海運 · 航運物流組 | AWS Summit Taipei 2026 百工百業瘋 AI — AI Everywhere Hackathon

團隊 FleetMind (工程 4 + 簡報 1)

THE PROBLEM

船殼污損每天在偷油——但何時該清，全靠經驗

船殼與螺旋槳污損讓同樣航速要燒更多油。船東靠水下清潔（UWC）與拋光（PP）恢復效能，但「何時做、值不值得、省多少」目前多憑經驗，缺乏可量化、可歸因、可回溯的決策依據。

15

艘船 · 5 年

匿名航行日報（noon report）

21,282

日報列

+ 77 筆水下養護事件

-12% ~ +22%

Speed Loss 全隊範圍

從剛養護到重度污損

102

格待預測油耗

S21-S23 養護後遮蔽窗格

兩條主線：看得見衰退，算得出代價

反事實預測

油耗預測模型

- 預測 102 格被遮蔽的全速油耗
- 反事實推論：現在做 UWC/PP 能少燒多少油
- 污損時鐘 + 跨姊妹船遷移學習

predict/ · Python · sklearn

效能診斷

Speed Loss Dashboard

- ISO 19030 框架下的效能衰退趨勢
- 船殼 vs 螺旋槳歸因
- 養護事件與效能恢復時序對比

core-calc + apps/api · Java / Spring Boot

兩者共用同一套確定性計算：多耗油量反事實、單服務可維運架構、AI 護欄，皆由此延伸

從噪音日報到可比的效能指標

1

載入 + 驗證

15 船日報 + 77 養護事件；型別 / 範圍檢查，不刪列只標旗標

2

事件直接對齊

maintenance.event_day 與日報 NOON.UTC 同軸，直接 join（無日曆錨點難題）

3

效能指標 k

$k = \text{Daily FOC} / \text{STW}^3$ ；以對水速度 STW 正規化，隔離洋流與航速

4

同速帶比較

參考窗 10-15 合格日、 ± 1 kn 同速帶、Theil-Sen 穩健趨勢

船隊結構

訓練船 S1-S12

全量可見

預測船 S21-S23

養護後遮蔽 · 102 格

W1 型 S1-S8, S21

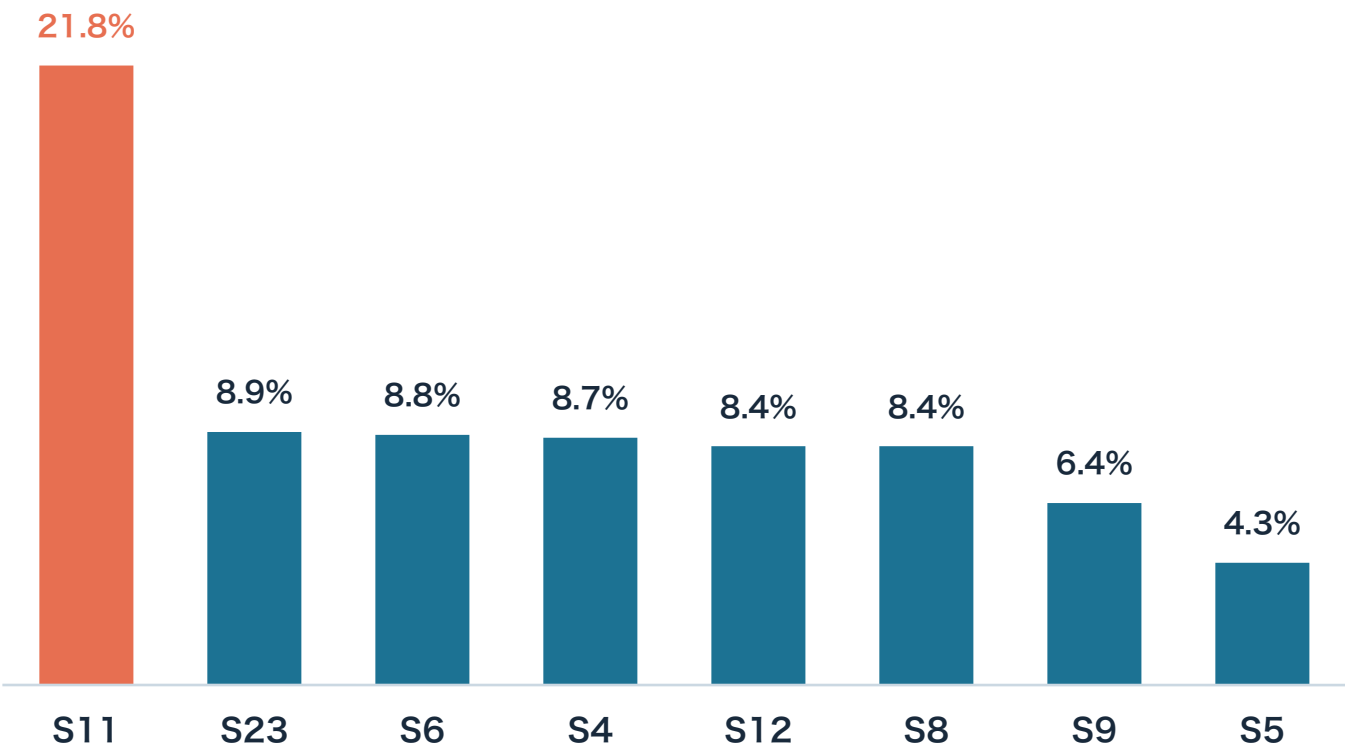
同設計姊妹船

W2 型 S9-S12, S22-S23

不同航線

被遮蔽的 3 艘預測船，在同型訓練船上有大量可見歷史 → 遷移學習。

船隊即時盤點：誰在漏油，先看誰



依 Speed Loss 排序的船隊優先盤點（前 8 名）

互動趨勢圖

點選任一船，看 Speed Loss 隨航程變化、事件標記、資料缺口

歸因卡

每船顯示船殼 / 螺旋槳佔比、信心等級與樣本數 n

Before-After

養護事件前後 median k 對比與恢復幅度

誠實信心

顯示 ± 區間、n、未解釋殘差，不做假精確

污損是船殼還是螺旋槳？—— 分開才好排養護

S23 範例

96%

Speed Loss 來自船殼污損

螺旋槳僅佔 4%

→ 建議優先安排水下清潔（UWC），拋光次之

方法：以隔離區段（只影響螺旋槳 / 只影響船殼的窗口）各自量 k 漂移率拆分；區段稀疏時退回標記過的 50/50，不假裝精準。

誠實處理「純檢查」事件（UWI）

官方提示：純檢查（UWI）不清不拋，不該帶來效能改善。

我們的做法 — 兩個地方，兩種角色：

- 預測模型：污損時鐘在 UWI 不重置 → 模型不會幻覺出一段恢復（物理先驗放這裡）。
- Dashboard：呈現實測 k 變化 + 信心，不宣稱「零變化」（稀疏資料上仍有雜訊，誠實顯示）。

在只給部分特徵下，還原被遮蔽的油耗

3.51

MT RMSE

5.22

% MAPE

102/102

格提交 1:1

演算法 物理 baseline ($k \cdot \text{STW}^3$) → 梯度提升樹 (HistGBM) → 依驗證擇優

特徵 STW/STW³/RPM/ 滑差 · 吃水載況 · 風浪湧水溫 · 污損時鐘 (UWI 不重置) · 船型 W1/W2 · 燃料熱值

防漏答驗證 在可見船上模擬真實遮蔽 (事件後合格日窗藏答案再評分) + 跨船 GroupKFold, 不用隨機 K-fold。

刻意的簡單

我們試過更複雜的堆疊與單調約束——在防漏驗證上都沒贏過樸素 GBM baseline, 於是資料驅動地選 baseline。

「不 ship 比 baseline 差的模型」是原則，不是能力上限。

提交標的：全速時段油耗總量 (MT)，以每小時率 × 全速時數還原。

同一個模型回答：污損正在多燒多少油？

反事實推論：把污損時鐘歸零重新預測，得到同航速下每天多耗的燃油噸數。只談噸數，不談金額。

粗估 · 非正式數字

船	可省比例	每日多耗燃油	信心
S23	24.4%	16.4 MT/ 日	HIGH · n=506
S6	25.1%	14.8 MT/ 日	LOW · 樣本不足
S11	52.3%	40.1 MT/ 日	極端 · 692 天未清

決策支援，非自動指令：低信心→先做便宜的水下檢查；高信心且多耗油量大→建議清潔；最終由輪機主管拍板。

誠實說限制，明確說要什麼

本次分析的限制

正午報表粒度

無軸功率、對水速度計等高頻量測； $k=FOC/STW^3$ 為 ISO 19030 的務實代理

洋流系統性偏差

SOG/STW 有差；有對水速度計可直接校正

歸因區段稀疏

部分船退回標記過的 50/50；需更多養護紀錄校準

UWI 統計雜訊

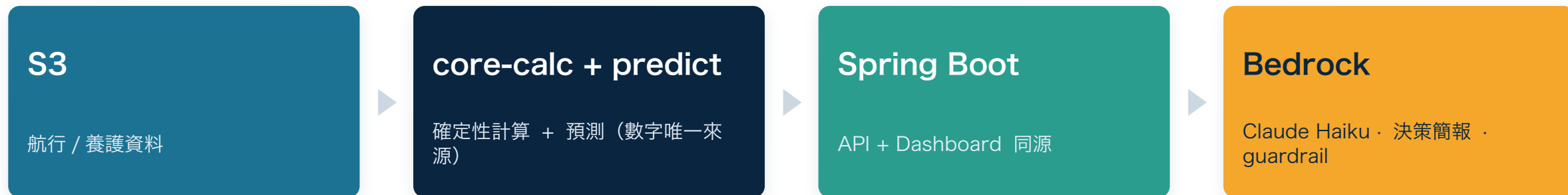
稀疏同速帶上仍有量測變化，誠實呈現而非壓平

給我們更多資料，能強化的決策價值

- 軸功率計 + 對水速度計 → 直接升級 ISO 19030 全合規
- 進塢後海試曲線 → 更準的效能參考基準
- 貴司 UWC/PP 恢復幅度紀錄 → 歸因與多耗油量估算有 ground truth
- 塗層年限 / 吃水俯仰 trim / 航線港口 → 更強的決策依據

官方必含 ④ 架構 + ⑤ AI 角色

單服務同源 · AI 只翻譯不發明



部署 ECS Fargate (ARM64) · ALB · ECR · Bedrock (Claude Haiku) · SNS · CloudWatch · us-east-1 · Day1 已實測上線

AI 的角色：數字來自計算，語言來自 AI，決策留給人

Bedrock(Claude) 只把已算好的指標改寫成營運語言；guardrail 逐一比對每個數字的引用來源，未引用或引錯即攔下——AI 無法發明數字。
失敗降級：Bedrock 不可用 → 該船快取簡報 → deterministic 模板；dashboard 與所有計算數字完全不受影響。開發全程以 Kiro 為 AI 助手。

FLEETMIND

把船隊的效能衰退，變成可計算、 可歸因、人可拍板的節能決策

Speed Loss Dashboard

真資料 15 船 · ISO 19030 · 船殼 / 螺旋槳歸因

油耗預測 102 格

RMSE 3.51 MT · 防漏驗證 · 反事實多耗油量

決策支援 · 人在迴路

AI 解釋不發明 · 單服務 AWS 架構

謝謝聆聽 · FleetMind × 陽明海運

資料與方法

Q 拿到什麼資料？怎麼跟養護事件對上？

A 15 船 × 5 年日報（21,282 列） + 77 養護事件。maintenance 用 event_day 與日報 NOON_UTC 同軸，直接 join，無日曆錨點問題。

Q 只有 3 艘要預測，樣本夠嗎？

A 船型 W1/W2 姊妹船；預測船在同型訓練船上有大量可見歷史 → 遷移學習，正是官方要的能力。

Q STW 還是 SOG？洋流怎麼處理？

A 阻力物理一律用 STW（對水）； $k = \text{FOC} / \text{STW}^3$ 正規化速度，只在 $\pm 1 \text{ kn}$ 同速帶比 k ，洋流不會被誤記成污損。

Q 預測的到底是什麼值？

A 當日全速時段主機油耗總量（MT，原欄位語義），非 24h 正規化。內部以每小時率 × 全速時數還原。

預測模型

Q 用什麼模型？效果多少？

A sklearn ；物理 baseline→HistGBM→擇優。最佳為樸素 GBM baseline，模擬遮蔽 RMSE 3.51 MT / MAPE 5.22%。更複雜的沒贏過 → 不上。

Q 怎麼確定沒偷看答案？

A 不用隨機 K-fold。在可見船上模擬真實遮蔽（事件後合格日窗藏答案再預測、逐窗評分）+ GroupKFold 確認跨船遷移。

Q RPM/SFOC 不是能反推油耗？算洩漏？

A H 類（SFOC/ 馬力 / 推力）在預測窗本就遮蔽、不用。ME_AVG_RPM 屬可見運轉條件、非答案代理；使用邊界也列入問主辦。

Q 模型有信心區間嗎？

A 每格附 $\pm$ band（同船型 × 燃料 slice 的殘差 std）；submission 主檔維持 4 欄 102 列，信心另存輔助檔。

Speed Loss / ISO 19030

Q 對齊 ISO 19030 哪個層級？

A noon-report 粒度務實改編，不宣稱全合規。k=FOC/STW³ 當 performance value 代理，骨架照 ISO：定參考期、控速帶、看偏移；偏離四點有明列表。

Q 全隊都降速了，怎麼分商業減速與污損？

A 從不比原始 FOC/ 航速。k 先正規化速度，只在同速帶比；基準取事件後首 10-15 合格日、上限 60 天。減速改變 V 不改變 k。

Q Speed Loss % 可信嗎？

A 段內對 k 做 Theil-Sen 穩健迴歸（抗離群）；主 KPI 顯示 3.5%（±0.8）· 中信心 · n=12，未解釋殘差一定顯示，不做假精確。

Q 船殼 vs 螺旋槳怎麼拆？

A 隔離區段各量 k 漂移率拆分；區段稀疏時退回標記過的 50/50，明講『因區段不足暫用預設』，不假裝精準。

UWI 判讀 · 商務價值

Q UWI 你說不改善，但圖上有變化？

A 兩個地方兩種角色：物理先驗放『預測模型污損時鐘不因 UWI 重置』；dashboard 誠實顯示實測 delta+ 信心，不硬壓成零。刻意分開統計實測與物理先驗。

Q 這系統能幫我省多少？講數字。

A 我們刻意不出金額。貴司工程師講得很清楚：成本是別部門管的，你們要的是「發現問題、通知需要清洗」。所以我們只講噸數——反事實把污損時鐘歸零重預測，S23 同航速下每天多耗約 16.4 MT（可省 24.4%，HIGH 信心 · n=506）。這是粗估、非正式數字；要換算成本，用貴司當期實際油價乘上去即可，每港每次加油價格都不同，我們不假裝知道。

Q AI 會自己排清潔嗎？

A 不會，決策支援、人拍板。低信心→先做便宜水下檢查；高信心且多耗油量大→建議清潔。反事實是檢視證據，不是自動指令。

Q 資料品質差的日子會硬給建議嗎？

A 被拒列標原因 + 品質分；可比樣本不足時不出高信心建議，改建議先做檢查。信心與 n 永遠跟著數字顯示。

AI · AWS · 限制

Q AI 扮演什麼角色？怎麼防亂編數字？

A 數字來自計算、語言來自 AI。Bedrock(Claude) 只改寫已算好的指標；guardrail 逐一比對每個數字的引用來源，未引用 / 引錯即攔——無法發明數字。開發用 Kiro。

Q 架構？成本？為何不上 SageMaker ？

A 單一 Spring Boot 容器：ECS Fargate（ARM64）+ ALB + ECR + Bedrock（Claude Haiku）+ SNS + CloudWatch，帳號 516665228894 · us-east-1，Day1 已實測上線。ARM64 Fargate 成本低、免管 EC2/OS patch。原本評估的 App Runner 本次受帳號權限阻擋（AccessDenied），故走 Fargate。不上 SageMaker：ISO 19030 本身確定性、樣本少硬煉會過擬合，複雜模型也沒贏過 baseline。

Q 最大的限制是什麼？（主動揭露）

A ① SOG 非對水速度、洋流系統偏差 ②無軸功率、k 是代理 ③歸因稀疏時退 50/50 ④UWI 仍有量測雜訊，呈現而非壓平。都在 UI/ 偏離表明示。

Q 給更多資料會怎麼強化？

A 軸功率計 + 對水速度計→升級 ISO 19030 全合規；海試曲線→更準基準；貴司 UWC/PP 恢復幅度紀錄→歸因與多耗油量估算有 ground truth。框架不改、餵更好量測即升級。